

Uvod u programiranje i dijagrami toka toka

Od problema do programa: algoritam, promjenljiva, operatori i dijagram toka

Programiranje u mehatronici

Svaki uređaj koji mjeri, odlučuje i djeluje, sadrži program. Učićemo da pišemo takve programe.

Primjer: Regulacija temperature

Očitava se senzor, upoređuje sa zadatom vrijednošću i uključuje grijač ili ventilator.

Primjer: Upravljanje motorom

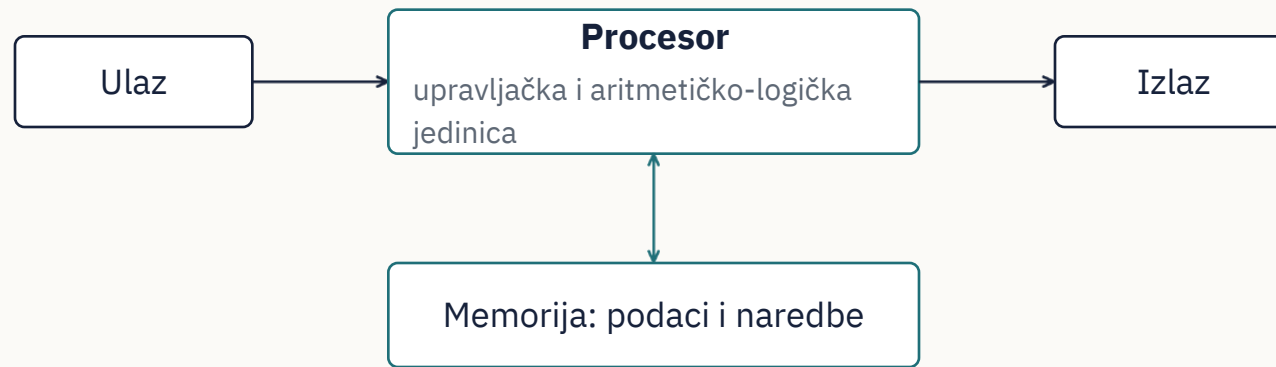
Računa se broj koraka i smjer obrtanja na osnovu zadate pozicije.

Primjer: Sigurnosno isključenje

Prati se granica pritiska ili struje i pogon se zaustavlja prije nastanka kvara.

Programski jezik C nastao je prije više od pedeset godina i njime se i danas pišu upravljački programi za mikrokontrolere i operativni sistemi. Pisan je tako da bude blizak mašini, pa ćemo zbog toga voditi računa o tipovima podataka i o memoriji.

Kako računar izvršava program

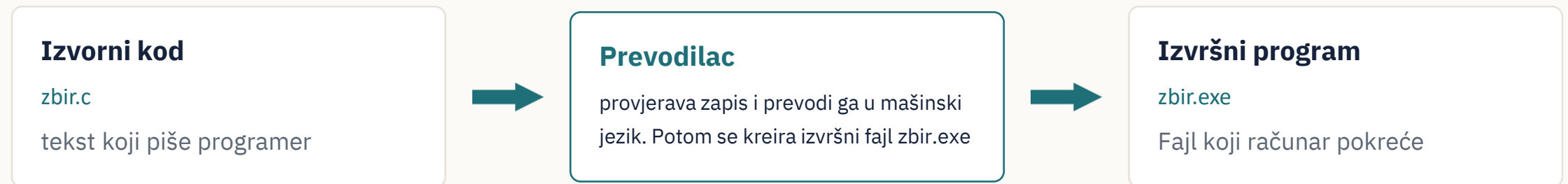


- Procesor izvršava **jednu po jednu naredbu programa**, po utvrđenom redoslijedu.
- Memorija čuva i **podatke** i same **naredbe** programa.
- Program uzima podatke sa ulaza i rezultat šalje na izlaz.
- Ulaz: tastatura, miš, senzor
- Izlaz: ekran, aktuator, robotska ruka

Zato se program piše kao **niz koraka sa jasnim redoslijedom**. Takav niz koraka nazivamo **algoritmom**.

Mašinski jezik i prevodilac

- Procesor razumije samo mašinski jezik, odnosno nizove brojeva koji označavaju naredbe.
- Programski jezici višeg nivoa postoje da bi pisanje programa bilo moguće i razumljivo čovjeku.
- Program koji izvorni kod prevodi u mašinski jezik naziva se prevodilac (engl. *compiler*).



U razvojnom okruženju CodeBlocks prevođenje i pokretanje programa pokreću se **jednim klikom**.



Razvojno okruženje

Na ovom kursu koristimo **Code::Blocks**. Besplatan je, sadrži i prevodilac, i instaliran je na svim računarima u sali.

1. File → New → Project → Console application → C
2. Naziv projekta i putanja bez razmaka i bez naših slova
3. Prevodilac: GNU GCC
4. Kod se piše u datoteci main.c
5. F9 prevodi program i pokreće ga

Naša slova u ispisu

U tekstu koji program ispisuje ne koristimo č, ć, š, ž.
Konzola ih podrazumijevano prikazuje pogrešno.

Prvi program u jeziku C

```
#include <stdio.h>

int main(void)
{
    int x, y, z;

    printf("Unesite dva cijela broja: ");
    scanf("%d %d", &x, &y);

    z = x + y;

    printf("Zbir je %d\n", z);

    return 0;
}
```

Izlaz

Unesite dva cijela broja: 7 5

Zbir je 12

Prvi red uključuje gotove funkcije za ulaz i izlaz, **printf** i **scanf**. Njih ne pišemo sami, one se već nalaze u standardnoj biblioteci jezika C.

Nakon prvog uspješnog pokretanja: zamijenite znak **+** znakom *****, pa ponovo prevedite i pokrenite.

- vitičaste zagrade su granice programa
- svaka naredba se završava tačkom-zarezom
- znak & ispred imena promjenljive kod scanf za sada uzimamo kao pravilo

Prva greška pri prevođenju

Obrišite tačku-zarez na kraju reda `z = x + y;` i pokrenite prevođenje.

```
z = x + y
```

```
printf("Zbir je %d\n", z);
```

Poruka prevodioca

zbir.c: In function 'main':

zbir.c:10:14: error: expected ';'

before 'printf'

```
10 | z = x + y
```

```
|      ^
```

Poruka prevodioca uvijek sadrži **broj reda**. Greška je u tom redu ili u redu iznad njega.

Faze u programiranju

1

Definicija problema

Šta je ulaz, šta je izlaz, šta mora važiti.

2

Algoritam

Koraci do rješenja, zapisani dijagramom toka.

3

Pisanje koda

Prepis algoritma u naredbe programskog jezika (u našem slučaju jezika C).

4

Prevođenje

Ispravljanje grešaka koje prijavi prevodilac (debugging).

5

Testiranje

Provjera na primjerima za koje rezultat znamo unaprijed.

Algoritam

DEFINICIJA

Algoritam je konačan niz jasno određenih koraka koji, primijenjen na ulazne podatke, dovodi do rješenja zadatog problema.

Primjer algoritma: priprema čaja

1. Sipaj vodu u posudu.
2. Zagrij vodu do ključanja.
3. Stavi kesicu u šolju.
4. Sipaj vodu u šolju.
5. Sačekaj tri minuta i izvadi kesicu.

Korak *dodaj šećer po ukusu* ne bi pripadao algoritmu jer nije jednoznačan, pa dvije osobe od njega dobijaju različit rezultat.

Ulaz i izlaz — ulaz su voda i kesica, izlaz je šolja čaja.

Konačnost — postupak se završava poslije petog koraka.

Određenost — svaki korak znači isto svakome ko ga čita.

Redoslijed — voda se ne sipa u šolju prije nego što je zagrijana.

Algoritam postoji nezavisno od programskog jezika. Pisanje programa je prepisivanje već osmišljenog algoritma u naredbe izabranog jezika.

Promjenljiva

DEFINICIJA

Promjenljiva je imenovano mjesto u memoriji u kojem se čuva jedna vrijednost. Ime ostaje isto, a vrijednost se može mijenjati tokom izvršavanja programa.

Izvršena naredba	vrijednost u promjenljivoj x	vrijednost u promjenljivoj y	vrijednost u promjenljivoj z
prije izvršavanja	—	—	—
x = 7	7	—	—
y = 5	7	5	—
z = x + y	7	5	12
z = z + 1	7	5	13

Znak jednakosti nije jednačina

Znak jednakosti u programiranju znači: izračunaj desnu stranu i upiši rezultat u promjenljivu sa lijeve strane.

Deklaracija

Promjenljiva se prije upotrebe deklarira i time se najavljuje kakve vrijednosti čuva: **int x;**

Možemo u jednom redu deklarirati više promjenljivih: **int x,y,z;**

Aritmetički operatori

Operator	Operacija	Primjer
+	Sabiranje	5 + 2 daje 7
-	Oduzimanje	5 - 2 daje 3
*	Množenje	5 * 2 daje 10
/	Cjelobrojno dijeljenje	11 / 2 daje 5
%	Ostatak pri dijeljenju	11 % 2 daje 1

Dijeljenje cijelih brojeva

Kada su oba operanda cijeli brojevi, rezultat dijeljenja je cio broj i decimalni dio se odbacuje: **11 / 2 daje 5**, a ne 5.5. Decimalni rezultat dobija se ako je bar jedan operand realan broj: **11.0 / 2 daje 5.5**.

Operatori poređenja

Operator	Značenje	Tačan primjer
>	Veće	7 > 5
<	Manje	3 < 5
>=	Veće ili jednako	5 >= 5
<=	Manje ili jednako	4 <= 9
==	Jednako	5 == 5
!=	Različito	5 != 3

Jedan ili dva znaka jednakosti

Poređenje se piše sa dva znaka jednakost: **x == 5**. Jedan znak, **x = 5**, znači **dodjelu** vrijednosti. Zamjena znaka za poređenje I znaka za dodjelu je česta greška programera i prevodilac je ne prijavljuje uvijek.

Rezultat poređenja

Poređenje daje odgovor tačno ili netačno. U jeziku C to su vrijednosti 1 i 0.

Logički operatori

Koriste se kada odluka zavisi od više uslova istovremeno.

&&

I — konjunkcija

Tačno samo ako su oba uslova tačna.

`t > 60 && rezim == 1`

||

ILI — disjunkcija

Tačno ako je bar jedan uslov tačan.

`t > 90 || p > 5`

!

NE — negacija

Mijenja tačno u netačno i obrnuto.

`!(t > 60)`

Primjer: ventilator se uključuje ako je temperatura veća od 60 stepeni i ako je uređaj u automatskom režimu.

Dijagram toka

Dijagram toka je **grafički prikaz** algoritma: pokazuje korake, njihov redoslijed i mjesta na kojima se izvršavanje grana.

Prikazuje tok, ne sintaksu

Vidi se šta se dešava poslije čega i gdje se izvršavanje dijeli na dva puta.

Nezavisan je od jezika

Isti dijagram prevodi se u C, u Python ili u program za mikrokontroler.

Crta se prije koda



Simboli dijagrama toka



Početak i kraj

Svaki dijagram ima tačno jedan početak i završava se krajem kojih može biti više.



Ulaz

Učitavanje podataka. Šira ivica je gore, kao posuda u koju se sipa.



Izlaz

Ispis rezultata. Šira ivica je dolje, kao okrenuta posuda.



Obrada

Izračunavanje i dodjela vrijednosti promjenljivoj.



Odluka

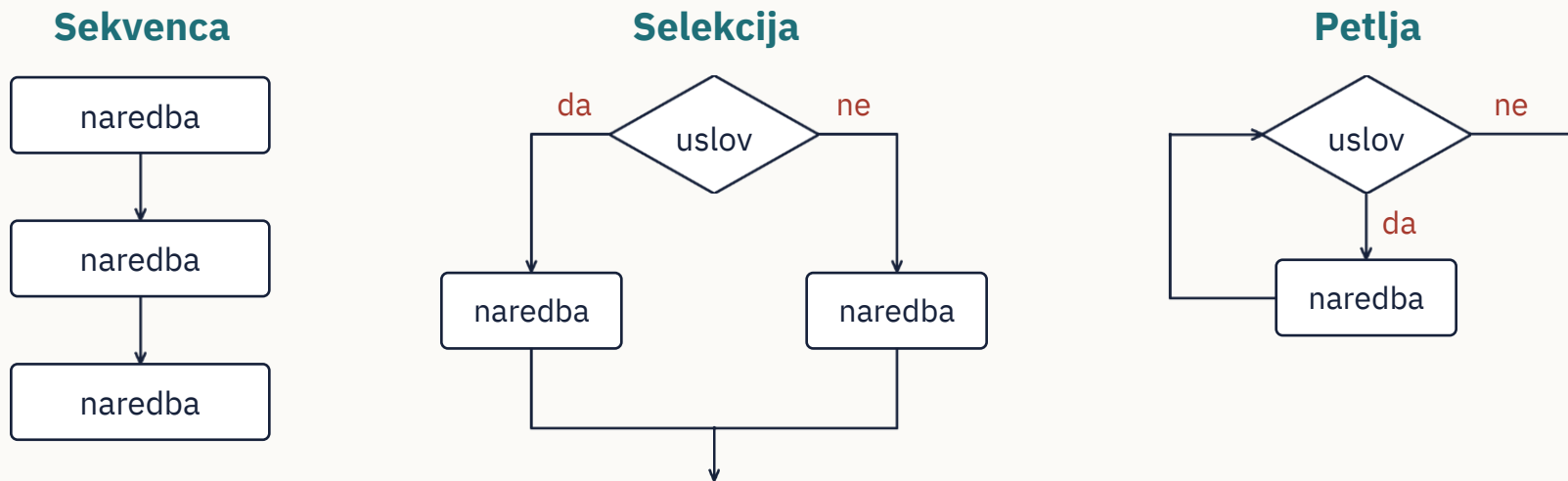
Uslov iz kojeg izlaze dvije grane: da i ne.



Tok izvršavanja

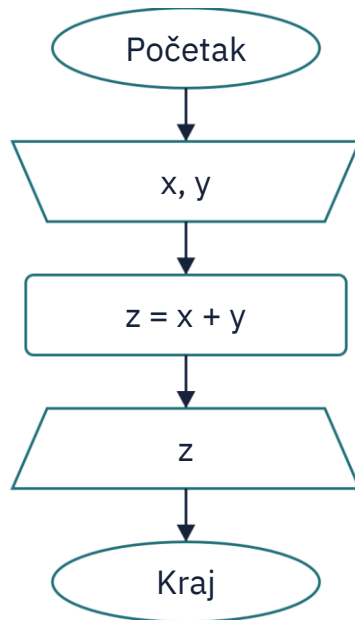
Svaka linija nosi strelicu koja pokazuje smjer.

Tri kontrolne strukture



Svaki program, bez obzira na složenost, sastavljen je isključivo od ove **tri strukture (kontrole toka)**.

Primjer 1: zbir dva broja

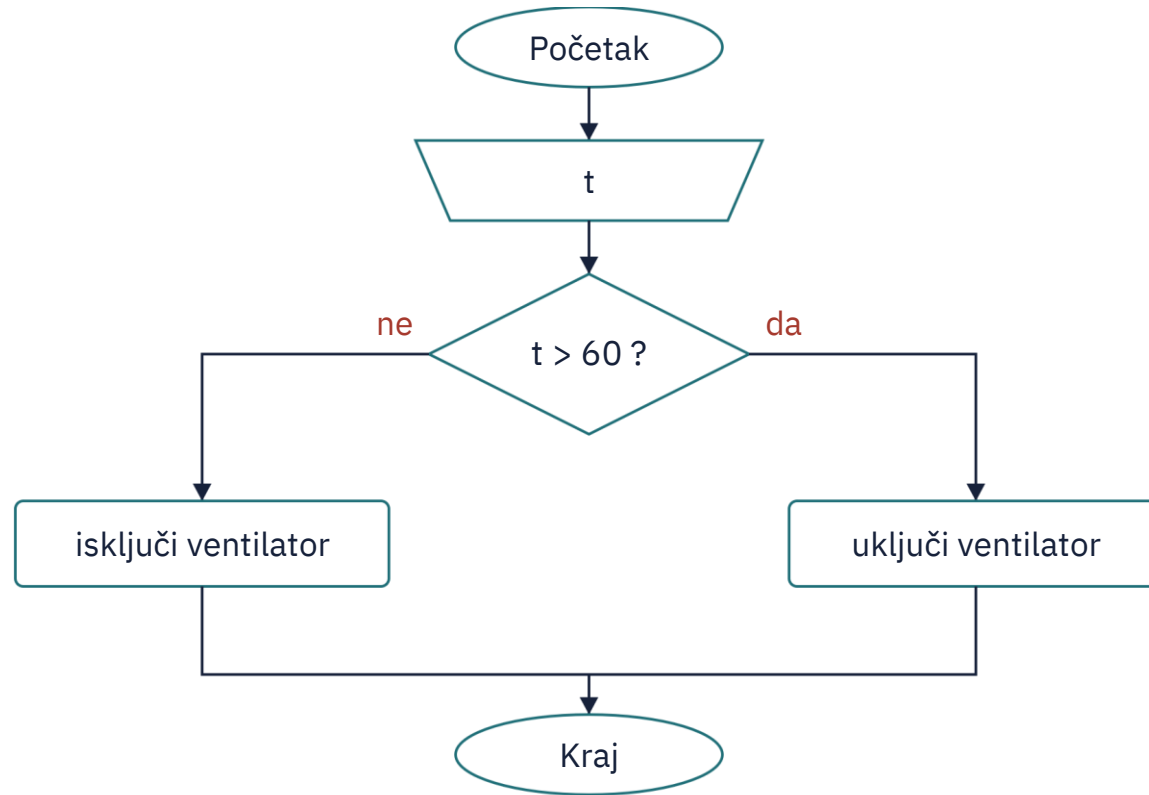


Primjer algoritma sastavljenog od sekvence prostih naredbi

```
int x, y, z;  
  
printf("Unesite dva broja: ");  
scanf("%d %d", &x, &y);  
  
z = x + y;  
  
printf("Zbir je %d\n", z);
```

Svaki simbol sa dijagrama toka postoji i u kodu.

Primjer 2: upravljanje ventilatorom



Primjer algoritma sa uslovnim grananjem (selekcijom)

Postavka

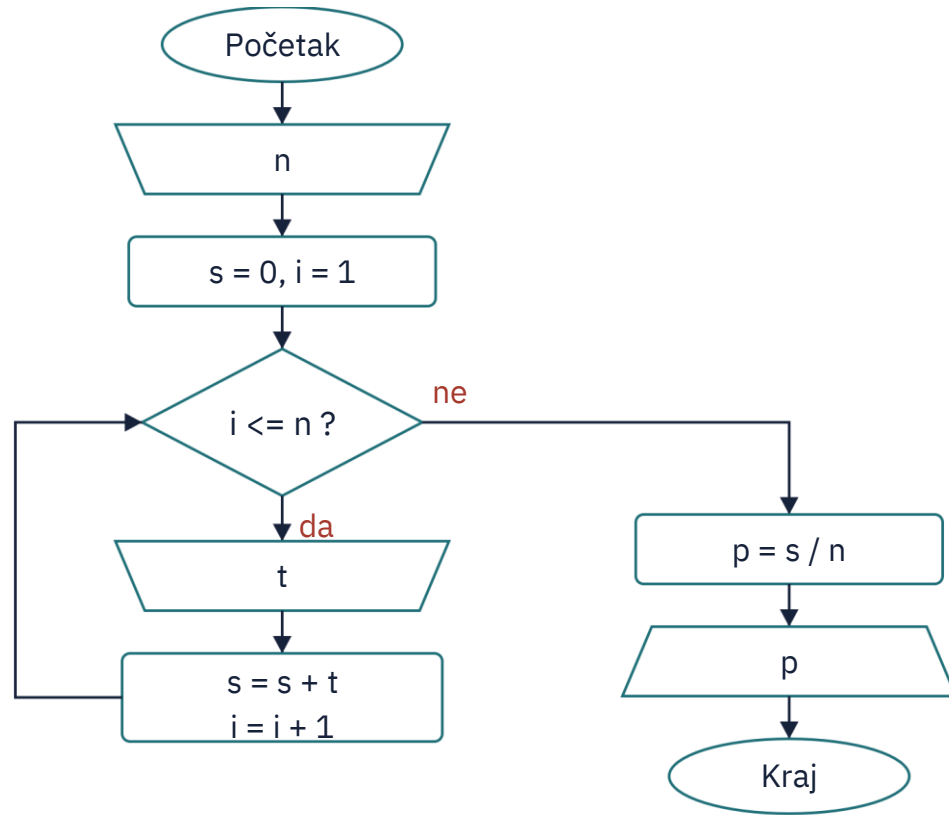
Uređaj očitava temperaturu. Ako je temperatura veća od 60 stepeni, uključuje ventilator; u suprotnom ga isključuje.

Iz romba izlaze obje grane. Obje se zatim spajaju i vode u isti kraj — program se ne završava na dva mjesta.

Program za ovaj dijagram pisaćemo kada uvedemo naredbu **if**

Primjer 3: srednja vrijednost n mjerenja

Primjer algoritma sa petljom



Postavka

Uređaj očitava n mjerenja temperature i računa njihovu srednju vrijednost.

s je zbir očitanih vrijednosti, i je brojač koji pamti koliko ih je do sada očitano. Obje dobijaju početnu vrijednost prije ulaska u petlju.

Bez reda $i = i + 1$ uslov bi zauvijek ostao tačan i petlja se ne bi zaustavila.

Četiri greške koje se često javljaju

Petlja bez izlaza (beskonačna petlja)

Uslov nikada ne postaje netačan. Provjerite da se unutar petlje **mijenja** ono od čega uslov zavisi.

Odluka sa jednim izlazom

Iz romba moraju izlaziti obje grane, i da i ne, i obje moraju negdje voditi.

Nepostavljena početna vrijednost

Brojač i zbir moraju dobiti vrijednost prije ulaska u petlju, a ne unutar nje.

Linije bez strelica

Bez strelice se redoslijed ne može pročitati, pa dijagram nije završen.

Prije nego što dijagram proglasite gotovim, prođite kroz njega sa konkretnim brojevima i zapišite vrijednosti promjenljivih u svakom koraku.

Zadaci za samostalan rad

crtaju se na papiru

Zadatak 1

Učitati dva broja i ispisati veći od njih.

Zadatak 2

Učitati broj i ispisati da li je paran.

Uputstvo: koristite operator %.

Zadatak 3

Ispisati brojeve od 1 do n.

Uputstvo: potrebni su brojač i petlja.

Zadatak 4

Uređaj mjeri temperaturu u pravilnim razmacima. Uključiti alarm ako tri uzastopna mjerenja pređu 80 stepeni.